Introduction to Computational Geometry

Borworntat Dendumrongkul

IOI Thailand Task Team

Outline

What is Computational Geometry?

Computational Geometry is the study of algorithms for solving problems defined in terms of geometric objects — points, lines, polygons, circles, ...

It sits at the intersection of:

- **Mathematics** — linear algebra, analytic geometry
- **Algorithm design** — correctness, complexity, numerical stability
- **Software engineering** — robust implementation under edge cases

Applications in Computer Science

Computational geometry appears everywhere in CS:

- **Computer Graphics & Games** — collision detection, visibility, ray tracing, mesh processing
- **Robotics & Motion Planning** — path finding, obstacle avoidance, workspace analysis
- **Geographic Information Systems (GIS)** — map overlays, spatial queries, routing
- **Computer Vision** — object recognition, point cloud processing, SLAM
- **VLSI / Circuit Design** — polygon clipping, design rule checking

Why Geometry is Hard to Implement

Geometric algorithms look simple on paper but are notoriously tricky in practice:

- **Edge cases everywhere** — parallel lines, collinear points, touching vs. crossing
- **Floating-point errors** — rounding can flip a sign and break the entire algorithm
- **Degenerate inputs** — three points on a line, zero-area polygon, duplicate points

A solution that works on 99% of inputs is *not* acceptable. Understanding the primitives deeply is what makes robust code possible.

Geometry in Competitive Programming

Geometry problems appear regularly in high-level contests:

- **IOI** — roughly 1–2 geometry or geometry-adjacent problems per year
- **ICPC** — geometry is a standard topic; teams without it lose a full problem
- **Codeforces / AtCoder** — geometry rounds and classic problems (convex hull trick, line sweep)

In Thailand's IOI selection process, geometry problems have historically appeared — being unprepared means giving up those points entirely.

The Competitive Programmer's Geometry Toolkit

What you actually need to solve most contest problems:

1. **Primitives** — points, vectors, lines, segments *(today)*
2. **Orientation test** — the backbone of nearly every algorithm *(today)*
3. **Convex hull** — classic and used as a sub-routine *(today)*
4. **Line segment intersection** — sweepline algorithms
5. **Polygon area, point-in-polygon, Minkowski sum** — ...

Start with the fundamentals — everything else builds on them.

Primitives

Primitives are the basic building blocks used in computational geometry:

- Points
- Vectors
- Line Segments
- Lines

Each primitive has its own properties and operations that are used to solve geometric problems.

Points

A point in 2D/3D space is represented as a coordinate tuple.

```
struct Point2D {
  double x, y;
};
struct Point3D {
  double x, y, z;
};
```

Vectors

A vector represents a **direction and magnitude**, not a specific location.

In 2D it shares the same structure as `Point2D` — we can inherit:

```
struct Vector2D : Point2D {}; // inherit from Point2D

// Vector from p1 to p2:
Vector2D v = {p2.x - p1.x, p2.y - p1.y};
```

Line Segments

A line segment is defined by two endpoints.

```
struct Segment {
  Point2D p1, p2;
};
```

Unlike a full line, a segment has a fixed start and end.

Lines — Two Common Representations

There are two main ways to represent a line in 2D:

- **Parametric Form** — point + direction vector
- **Implicit Form** — equation $Ax + By + C = 0$

Lines — Parametric Form

$$L(t) = P + t \cdot \vec{d}$$

where P is a base point and $\vec{d}$ is the direction vector.

```
struct ParametricLine {
  Point2D P;
  Vector2D d;
};
```

Lines — Implicit Form

$$Ax + By + C = 0$$

For a line through $p_1(x_1, y_1)$ and $p_2(x_2, y_2)$:

$$A = y_1 - y_2$$
$$B = x_2 - x_1$$
$$C = -(A \cdot x_1 + B \cdot y_1)$$

```
struct ImplicitLine {
  double A, B, C;
};
```

Dot Product

Given $\vec{a} = (a_x, a_y)$ and $\vec{b} = (b_x, b_y)$:

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$$

Geometric interpretation:

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta$$

- > 0: angle between vectors is acute
- $= 0$: vectors are perpendicular
- < 0: angle is obtuse

Cross Product

Given $\vec{a} = (a_x, a_y)$ and $\vec{b} = (b_x, b_y)$, extend to 3D with $z = 0$:

$$\vec{a} \times \vec{b} = \langle 0,\ 0,\ a_x b_y - a_y b_x \rangle$$

The **scalar** $a_x b_y - a_y b_x$ is what we use in 2D geometry.

- > 0: $\vec{b}$ is to the **left** of $\vec{a}$ (counter-clockwise)
- $= 0$: vectors are parallel (collinear)
- < 0: $\vec{b}$ is to the **right** of $\vec{a}$ (clockwise)

Dot Product & Cross Product — Code

```
double dot(Vector2D a, Vector2D b) {
  return a.x * b.x + a.y * b.y;
}

double cross(Vector2D a, Vector2D b) {
  return a.x * b.y - a.y * b.x;
}
```

Orientation Test

Given three points p_1, p_2, p_3, determine whether they are arranged:

- **Counter-Clockwise (CCW)**
- **Clockwise (CW)**
- **Collinear**

This is one of the most fundamental operations in computational geometry.

Naive Approach — Slopes

Compare the slope of segment (p_1, p_2) with the slope of (p_1, p_3):

$$\text{slope}_{12} = \frac{p_2.y - p_1.y}{p_2.x - p_1.x}$$

Problem: division by zero when $p_2.x = p_1.x$ (vertical segment).

Also susceptible to floating-point errors.

Better Approach — Cross Product

Compute the cross product of vectors $(p_2 - p_1)$ and $(p_3 - p_1)$:

$$\text{orientation} = (p_2.x - p_1.x)(p_3.y - p_1.y) - (p_2.y - p_1.y)(p_3.x - p_1.x)$$

Equivalently, this is the **determinant**:

$$\det \begin{pmatrix} x_2 - x_1 & y_2 - y_1 \\ x_3 - x_1 & y_3 - y_1 \end{pmatrix}$$

(= twice the signed area of the triangle $p_1 p_2 p_3$)

Interpreting the Result

- orientation > 0: p_1, p_2, p_3 are **Counter-Clockwise (CCW)**
- orientation < 0: p_1, p_2, p_3 are **Clockwise (CW)**
- orientation $= 0$: p_1, p_2, p_3 are **Collinear**

Orientation — Code

```
int orientation(Point2D p1, Point2D p2, Point2D p3) {
  double v = (p2.x - p1.x) * (p3.y - p1.y)
           - (p2.y - p1.y) * (p3.x - p1.x);
  if (v > 0) return  1; // Counter-Clockwise
  if (v < 0) return -1; // Clockwise
  return 0;             // Collinear
}
```

Floating Point Errors

Computers store floating-point numbers in **binary**, e.g.:

$$0.1 = 0.0001100110011\ldots_2$$

The binary expansion is truncated, introducing rounding errors.

So the computer does *not* store exactly 0.1 — it stores something like `0.10000000000000001` or `0.09999999999999999`.

IEEE 754 Standard

A `double` stores: Sign $\times$ Mantissa $\times$ 2^{Exponent}

- Mantissa: 52 bits
- Exponent: 11 bits
- Gives approximately 15–17 significant decimal digits

Subtracting two nearly-equal values can cause **catastrophic cancellation**, drastically reducing precision.

Solution 1 — Integer Arithmetic

If all coordinates are integers, avoid floating point entirely by using `long long`:

```
int orientation(Point2D p, Point2D q, Point2D r) {
  long long v = (long long)(q.x - p.x) * (r.y - p.y)
              - (long long)(q.y - p.y) * (r.x - p.x);
  if (v > 0) return  1;
  if (v < 0) return -1;
  return 0;
}
```

Solution 2 — Epsilon Comparison

Instead of comparing to exactly 0, use a small tolerance ε:

```
const double EPS = 1e-9;

int orientation(Point2D p, Point2D q, Point2D r) {
  double v = ...;
  if (v >  EPS) return  1;
  if (v < -EPS) return -1;
  return 0;
}
```

Caveats of Epsilon

Choosing ε is not straightforward:

- ε **too large**: cases that should be "not equal" become "equal"
- ε **too small**: does not actually fix floating-point errors

An alternative is **relative epsilon**: $\varepsilon \cdot \max\{|x|, |y|\}$,
but this breaks down when values are very close to zero.

Solution 3 — Exact Arithmetic

Replace `double` with **exact rational numbers** stored as integer pairs (numerator / denominator).

- No rounding at all — results are always exact
- Numbers can grow very large (requires big integers or careful bounding)
- Slower than floating point, but correct

Convex Hull

The **convex hull** of a set of points is the smallest convex polygon that encloses all the points.

Think of it as stretching a rubber band around all the points and letting it snap tight.

We will cover three algorithms:

1. Naive $\mathcal{O}(n^3)$
2. Jarvis March (Gift Wrapping) $\mathcal{O}(nh)$
3. Graham Scan / Monotone Chain $\mathcal{O}(n \log n)$

Naive Algorithm

1. Consider every pair of points (p, q) — there are $\mathcal{O}(n^2)$ pairs.
2. For each pair, check whether all other $n-2$ points lie on the **same side** of the line through (p, q). This takes $\mathcal{O}(n)$ per pair.
3. If yes, edge (p, q) is on the convex hull.

Time Complexity: $\mathcal{O}(n^3)$

Jarvis March (Gift Wrapping)

Simulate *wrapping a gift* around the point set:

1. Start from the point with the smallest y-coordinate (guaranteed on hull).
2. Repeatedly pick the point that makes the **smallest clockwise angle** with the current direction — i.e., the most counter-clockwise point from the current one.
3. Stop when we return to the start.

Time Complexity: $\mathcal{O}(nh)$, where h is the number of hull vertices.
Optimal when h is small; worst case $\mathcal{O}(n^2)$.

Graham Scan — Overview

Key idea: reduce the 2D hull problem to a 1D sorting problem.

1. **Sort:** pick a pivot (lowest y), sort remaining points by polar angle around pivot.
2. **Scan:** sweep through sorted points, maintaining a stack. Pop any point that makes a **clockwise** (right) turn; push otherwise.

Time Complexity: $\mathcal{O}(n \log n)$ (dominated by the sort).

Angle Sorting — The Problem

Sorting by polar angle naively uses arctan:

$$\theta = \arctan\left(\frac{y_2 - y_1}{x_2 - x_1}\right)$$

Problems:

- Division by zero when $x_2 = x_1$ (vertical direction)
- Floating-point errors from trigonometric functions

We don't actually need the angle — we only need the **order**.

Angle Sorting — Using Orientation

Key insight: For pivot P and two points A, B:

A comes before B in CCW polar order $\iff$ `orientation(P, A, B)` > 0

If they are **collinear** (`orientation` $= 0$), put the **closer** point first:

$$\text{dist}^2(P, A) = (A.x - P.x)^2 + (A.y - P.y)^2$$

No division, no trigonometry — purely integer arithmetic is possible.

Monotone Chain

An alternative to Graham Scan that avoids angle sorting entirely:

1. **Sort** points lexicographically by (x, y).
2. Build the **lower hull**: sweep left $\rightarrow$ right, pop CW turns.
3. Build the **upper hull**: sweep right $\rightarrow$ left, pop CW turns.
4. Concatenate lower and upper hull (remove duplicate endpoints).

Time Complexity: $\mathcal{O}(n \log n)$

Simpler to implement correctly than Graham Scan.

Bibliography I

- Andrew, A. M. (1979). "Another Efficient Algorithm for Convex Hulls in Two Dimensions". In: *Information Processing Letters*. Vol. 9. 5. Monotone Chain algorithm. Elsevier, pp. 216–219.
- Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. Chapter 33: Computational Geometry. MIT Press. ISBN: 978-0-262-03384-8.
- Cp algorithms.com (2024). *Competitive Programming Algorithms — Geometry*. https://cp-algorithms.com/geometry/. Accessed: 25 March 2026.
- De Berg, Mark et al. (2008). *Computational Geometry: Algorithms and Applications*. 3rd. Springer. ISBN: 978-3-540-77973-5.
- Graham, Ronald L. (1972). "An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set". In: *Information Processing Letters* 1.4, pp. 132–133.
- IEEE (2019). *IEEE Standard for Floating-Point Arithmetic*. Institute of Electrical and Electronics Engineers.

Bibliography II

- Jarvis, R. A. (1973). "On the Identification of the Convex Hull of a Finite Set of Points in the Plane". In: *Information Processing Letters* 2.1, pp. 18–21.
- Of Technology, KTH Royal Institute (2024). *KACTL: KTH Algorithm Competition Template Library*. https://github.com/kth-competitive-programming/kactl. Reference implementation for competitive programming geometry. Accessed: 25 March 2026.
- O'Rourke, Joseph (1998). *Computational Geometry in C*. 2nd. Cambridge University Press. ISBN: 978-0-521-64976-6.
- Preparata, Franco P. and Michael Ian Shamos (1985). *Computational Geometry: An Introduction*. Springer. ISBN: 978-0-387-96131-6.
- Skiena, Steven S. (2020). *The Algorithm Design Manual*. 3rd. Springer. ISBN: 978-3-030-54255-9.